Get Hired Again: Java Backend + Cloud + AI, 2026–2027

Prepared October 7, 2026 for Mohammed. Assumes about 12 hours a week of software-engineering study on top of McDaniel.

1. The plan on one page

Target role. Java and Spring Boot backend engineer at banks, fintechs and enterprises (NJ/NYC or remote), growing into AI-enabled backend work. Not UI full-stack, not ML research.

Honest timeline. An offer before the end of 2026 is a stretch given the gaps you listed. The realistic target is offers between February and May 2027, with applications going out in stages from now so you get interview practice early. If a contract or interview shows up sooner, the plan has an interview-mode switch (section 10).

Why this order. Interview loops gate on three things: data structures and algorithms, Java concurrency and JVM fundamentals, and system design. Resume filters gate on Spring Boot, Docker and Kubernetes, Kafka and AWS. AI is the differentiator, not the door-opener. So the plan spends the first month on tools, then Java depth and databases, then system design and Kafka, then cloud, then AI, with DSA running underneath all of it.

How you learn. You are not a beginner, so nothing here is “finish the course, then build”. Every course exists to unblock the next step of one capstone repository, and the rule is no course chapter without a commit. Skim what you know, slow down where the build breaks.

Weekly budget (about 12 hours). DSA 4 h, courses 3 h, capstone and labs 4 h, job search and mock interviews 1 h. If your real number is 8 hours, stretch every phase by 1.5. If it is 16, compress by about a quarter. In a McDaniel crunch week, run the minimum week: 3 DSA problems and 2 hours on the capstone, nothing else, no guilt.

Phases at a glance

- Phase 1, Weeks 1–4 (Oct to early Nov): Linux, Git, Docker, project setup
- Phase 2, Weeks 5–10 (Nov to mid Dec): Java concurrency and databases
- Phase 3, Weeks 11–16 (mid Dec to late Jan): system design, Kafka, Kubernetes
- Phase 4, Weeks 17–24 (late Jan to late Mar): AWS and deploying the capstone

- Phase 5, Weeks 25–32 (late Mar to May): AI engineering and polish
- Running throughout: DSA track, capstone commits, job-search track

2. Your toolkit and what each is for

Coursera Plus is your structured-theory and lab platform: Linux, Git, Docker and Kubernetes labs, AWS orientation, algorithms, and the DeepLearning.AI AI courses. Where Coursera is weak (concurrency, system design, Kubernetes depth), the plan says so and points elsewhere.

Pluralsight covers short, current courses: Spring Boot 4, PostgreSQL, Kubernetes for developers, FastAPI.

Udemy through your library card covers Java concurrency, Kafka and AWS exam prep. Library catalogs vary, so each Udemy pick has an alternative on Coursera or Pluralsight if it is not available to you.

GitHub Student Developer Pack gives you JetBrains tools (IntelliJ IDEA Ultimate is the one you want for Spring), GitHub Pro, Codespaces, LocalStack (an AWS emulator for local practice) and some cloud credits. Copilot Student exists, but GitHub paused new sign-ups for it earlier in 2026, so check your account rather than planning around it. Offers change, so confirm on the pack page.

Gemini AI Pro, ChatGPT Go and NotebookLM are tutors, interviewers and note organizers, not code authors. The rules are in section 7.

3. Weekly rhythm

Mon, Tue, Thu: 1 h DSA plus 1 h course. Wed: 1 h DSA plus 1 h capstone. Sat: 3 h capstone or lab. Sun: 1 h mock interview or system-design drill, then send me the weekly report (section 9). That is about 12 hours and leaves one weekday free for McDaniel overflow. Move blocks around, but keep the four DSA hours.

4. The roadmap

Phase 1 — Tools and foundations (Weeks 1–4)

Goal. Be comfortable in a Linux terminal, use Git the way teams do, run your own services in Docker, and have the capstone repo and your public profiles ready.

Learn

- Docker: finish the Packt Docker course you started today (Coursera, included with Plus). Practice in Play with Docker (free browser labs) and by containerizing your own app. Do not start a second Docker course.
- Linux: “Foundations of Linux” (Packt, Coursera, about 7 hours, part of the CompTIA Linux+ specialization), then “Linux Command Line” (Packt, Coursera, about a week). For the interactive side, play OverTheWire Bandit (free) and work through Linux Journey (free). Targets: permissions, processes, grep/sed/awk basics, bash scripting, ssh, systemd, and network tools such as ss and curl.
- Git and GitHub: “Introduction to Git and GitHub” (Google, Coursera, about 20 hours, so skim the early modules and spend your time on branching, pull requests and code review), plus Learn Git Branching (free, interactive) and GitHub Skills (free).
- Java refresh: Pluralsight “What’s New in Java SE 25” (about 1 hour) and “Spring Boot 4 Fundamentals” (about 2 hours). Your uploaded roadmaps list JDK 25 as the latest LTS and Spring Boot 4 as current; employers still run 17 and 21, so be able to read all three eras of code.

Build

- Create the capstone repository: a Spring Boot REST skeleton, a multi-stage Dockerfile, a docker-compose file with PostgreSQL, and a GitHub Actions workflow that runs the tests.
- Work only through branches and pull requests, even alone. Review your own PRs.
- Three bash scripts in a linux-labs folder: a log parser, a backup-and-rotate script, a service health check.

DSA focus. Arrays and hashing, two pointers, sliding window, stack.

Gate 1 (end of Week 4). `docker compose up` runs the stack from a clean clone. Resume, LinkedIn and GitHub profile refreshed. About 20 DSA problems done. Start sending about 5 applications a week to calibrate the market and get recruiter conversations, even though you are not at full strength yet.

Phase 2 — Java depth: concurrency and databases (Weeks 5–10)

Goal. Close the two biggest interview gaps for a Java backend role and make the capstone a real data-backed service.

Learn: concurrency

- Udemy “Java 25: Virtual Threads, Concurrency Masterclass [Hands-On]” (updated February 2026; threads, executors, CompletableFuture, virtual threads, structured concurrency, and a Spring Boot migration project). Weeks 5–7. If you prefer a named instructor and a more foundational treatment, swap in Michael Pogrebinsky’s “Java Multithreading, Concurrency & Performance Optimization” (Udemy).
- Book, as a slow companion: Java Concurrency in Practice (Goetz et al.), chapters 2–6, 8, 10–13 and 15–16.
- Optional Coursera theory: “Concurrent Programming in Java” (Rice University, Vivek Sarkar; course 2 of the Parallel, Concurrent, and Distributed Programming in Java specialization). Skip it if you are short on time; the Udemy course plus the book is enough for interviews.

Learn: databases

- Pluralsight: “Querying Data from PostgreSQL” (about 1.5 h), “Modify Data with SQL in PostgreSQL” (about 1.8 h, covers transactions and isolation) and “PostgreSQL Performance Tuning and Security” (about 2 h).
- CMU 15-445/645 Intro to Database Systems (Andy Pavlo, free on YouTube, 26 lectures of about 75 minutes). Watch only the lectures on storage and the buffer pool, B+tree indexes, query execution and optimization, concurrency control, and logging and recovery, roughly a dozen. Skip the projects.
- “Use The Index, Luke” (Markus Winand, free online): indexing from a developer’s point of view.

Build

- Capstone: PostgreSQL schema for documents and metadata, JPA with explicit transaction boundaries, migrations, and tests against a real database in a container.
- Concurrency lab: reproduce a lost-update race, fix it with synchronized, AtomicLong and LongAdder, build a producer-consumer pipeline with a BlockingQueue, create and diagnose a deadlock from a thread dump, then load-test platform threads against virtual threads and write up the numbers.
- Database lab: seed a million rows, run EXPLAIN ANALYZE before and after adding an index, and demonstrate an isolation anomaly with two psql sessions.

DSA focus. Linked lists, trees, heaps, tries, backtracking.

Gate 2 (Week 10). You can explain volatile versus atomic versus synchronized and happens-before without notes, and read an EXPLAIN plan. About 60 DSA problems. Capstone MVP with tests. Raise applications to 10–15 a week and start asking for

referrals.

Phase 3 — System design, Kafka and Kubernetes (Weeks 11–16)

Goal. Be able to design a service under load and talk about failure, then add the event-driven and container-orchestration pieces that appear in most job descriptions.

Learn: system design

- Book: Designing Data-Intensive Applications (Martin Kleppmann), chapters 5–9 first (replication, partitioning, transactions, consistency and consensus).
- Framework and worked designs: System Design Interview, Volume 1 (Alex Xu) or the ByteByteGo material. On YouTube use Hello Interview, ByteByteGo and Gaurav Sen, and attempt each design yourself before watching. DesignGurus' Grokking the System Design Interview is an optional paid question bank.
- Drill: one timed 45-minute design a week, spoken aloud and recorded, with an AI model playing the interviewer. Rotate through a URL shortener, rate limiter, notification service, payments ledger, event-driven order pipeline and document search.

Learn: Kafka

- Udemy "Apache Kafka Series: Learn Apache Kafka for Beginners v3" (Stéphane Maarek, about 8.5 hours, Java producers and consumers). It was released in 2022, so confirm it covers KRaft mode and fill gaps from the free Confluent Developer "Kafka 101".
- Alternative that fits Spring directly: Udemy "Spring Boot + Apache Kafka: The Practical Guide" (updated September 2026, about 5.5 hours).

Learn: Kubernetes at developer level

- Pick one: the Pluralsight "Kubernetes for Software Development" path (5 courses and a lab, about 10 hours), or Coursera "Introduction to Containers w/ Docker, Kubernetes & OpenShift" (IBM, about 17 hours with hands-on labs; skip the OpenShift module). Practice on kind or minikube locally and on Killercoda (free browser scenarios). You need Pods, Deployments, Services, ConfigMaps, Secrets, probes and rollouts, not cluster administration.

Build

- Write a one-page design doc for the ingestion pipeline before coding it.

- Move the capstone to events: DocumentUploaded to TextExtracted to EmbeddingsCreated to AnswerProduced, with idempotent consumers, a dead-letter topic and the transactional outbox pattern.
- Deploy the whole stack to a local Kubernetes cluster with a Deployment, a Service, a ConfigMap and a readiness probe.
- Failure drill: kill a consumer mid-batch and write down what happened and why.

DSA focus. Graphs and 1-D dynamic programming.

Gate 3 (Week 16, late January). Six timed designs completed, about 100 DSA problems, a Kafka flow and Kubernetes manifests in the repo, and a design doc. Move to 15–25 targeted applications a week and one mock interview every week. Hiring typically picks up after the holidays, which is why this gate sits here.

Phase 4 — Cloud: AWS (Weeks 17–24)

Goal. Tick the cloud box with something you actually deployed, not just a certificate.

Learn

- Coursera “AWS Cloud Technical Essentials” (Amazon Web Services; four modules) for fast orientation in Weeks 17–18. It was designed for learners with a technical background.
- Udemy “Ultimate AWS Certified Solutions Architect Associate” (Stéphane Maarek, about 27 hours, SAA-C03) in Weeks 18–22. If you want everything inside Coursera, use the “AWS Certified Solutions Architect – Associate Specialization” (Total Seminars, about 31 hours) or the shorter Packt SAA-C03 specialization (about 23 hours).
- Practice exams: Tutorials Dojo (Jon Bonso), at least two full exams before you book. This is a separate purchase.
- Skip the Cloud Practitioner exam unless an employer asks for it; the Technical Essentials course gives you the vocabulary, and the Associate exam is the one with real signal for engineers.
- Infrastructure as code: HashiCorp’s free Terraform tutorials, enough for a VPC and an ECS service. Use LocalStack from the Student Pack to practice without spending money.

Build

- Deploy the capstone to AWS: ECS Fargate with RDS PostgreSQL in private subnets, least-privilege IAM roles, Secrets Manager and CloudWatch logs, defined in Terraform or CloudFormation. Set an AWS Budgets alarm and tear the stack down after demos.

- Sit the SAA-C03 around Week 24. Check the current exam fee and policies on the AWS certification site before you book.

DSA focus. Intervals, greedy, 2-D DP, then review.

Gate 4 (late March). A live deployed demo with an architecture diagram, SAA-C03 passed or booked, about 130 DSA problems with review. Offers should start landing in this window if application volume held.

Phase 5 — AI engineering and polish (Weeks 25–32)

Goal. Add the differentiator: an AI service you can explain end to end, evaluated with numbers.

Learn

- Coursera “Generative AI with Large Language Models” (DeepLearning.AI and AWS, about 16 hours; free to audit).
- Coursera “Retrieval Augmented Generation (RAG)” (DeepLearning.AI). Do it before building.
- “Building and Evaluating Advanced RAG” (DeepLearning.AI short course, about 1 hour): evaluation is the part most portfolios skip.
- Python backend: Pluralsight’s FastAPI path, foundation and async sections only.
- McDaniel overlap: your Master’s coursework (Python, pandas, statistics, machine learning) supplies the ML foundations, so do not buy a second machine-learning course. Reuse its datasets for evaluation if the program permits sharing the work.

Build

- A Python FastAPI service that chunks documents, creates embeddings, stores them in PostgreSQL with pgvector and serves hybrid search. The Java API reaches it through Kafka or REST.
- An evaluation harness on a fixed question set that scores retrieval quality and answer quality, and records latency and cost. Put the numbers in the README.
- If you are already hired by now, continue this part-time; it becomes your growth story at the new job.

5. The DSA track (runs the whole time)

Anchor. NeetCode 150 in Java (neetcode.io), one pattern at a time. For the theory behind the patterns, use Princeton “Algorithms, Part I” (Robert Sedgewick and Kevin Wayne, Coursera, 13 modules, about 5 weeks at 10 hours a week). Watch only the lectures as you reach each topic: analysis of algorithms, union-find, stacks and queues, sorting, priority queues, symbol tables, BSTs and hashing. Skip the autograded assignments except Percolation, once, for depth. Part II (graphs and strings) is optional later.

Method. Attempt cold for 25 minutes, then take a hint, then read a solution, then re-solve it a week later. Keep a mistakes log. Talk through your reasoning out loud in Java, because that is the interview.

Pace. About 5–6 problems a week with review, around 130 by Week 24. Do not chase 150 at the expense of review.

Mocks. From Week 12, one timed mock a week: two mediums in 45 minutes, spoken aloud.

6. The capstone repository

One repo grows through every phase: a financial-document intelligence service. Users upload documents; the system extracts text, chunks and embeds them, indexes the content, answers questions with citations, and exposes audit information. The build order maps to the phases: skeleton and Docker (Phase 1), PostgreSQL and a concurrent ingestion path (Phase 2), Kafka events and Kubernetes (Phase 3), AWS deployment (Phase 4), AI service and evaluation (Phase 5).

Publish this evidence, because the README is what interviewers read: an architecture diagram and a two-to-three page design document, local setup with Docker Compose and API examples, a short performance note (baseline versus cached latency and a bottleneck analysis), a failure write-up showing how logs and retries helped you diagnose a problem, and the AI evaluation report.

7. Using Gemini, ChatGPT and NotebookLM without cheating yourself

- Try first for 20 minutes. Then ask for a hint, not the answer.
- Ask the AI to review your code and attack your design, not to write it. If you cannot explain a line you pasted, rewrite it.

- Use it as the system-design and behavioral interviewer, with a prompt such as: “Act as a senior interviewer. Give me a 45-minute design problem, ask clarifying questions only when I ask, then critique my answer on requirements, scale, bottlenecks and failure modes.”
- Keep one NotebookLM notebook per topic (Linux and Docker, concurrency, databases, system design, AWS) with your own notes and the official docs, and quiz yourself from it.

8. The job-search track

Profiles first. A one-page resume that leads with outcomes and numbers from your engineering experience, a clean LinkedIn, and a GitHub profile whose pinned repo is the capstone. Prepare six behavioral stories in STAR form, including your path from civil engineering to software to data and AI; it is a strength when framed deliberately.

Volumes by gate. About 5 applications a week from Week 4, 10–15 from Week 10, 15–25 from Week 16, with weekly mocks. Track company, date, referral, stage and outcome in a sheet.

Channels. Referrals first (LinkedIn, McDaniel and past colleagues), then recruiters and staffing firms for contract roles, which often move faster than direct hiring, then direct applications to banks and fintechs.

Weekly interview loop. Two coding problems, one system-design sketch, one concurrency or database debugging question, and one behavioral story.

9. Check-ins with me

Send a short report each week with four lines (Built, Applied, Stuck on, Next) plus three numbers: DSA problems solved, applications sent, mocks done. At each gate, send the repo link, the current README, a paragraph on what surprised you, and what you would cut. I will review the gap, adjust the next phase and push back on your design decisions.

10. Risks and what to cut first

McDaniel crunch. Run the minimum week and keep going.

Interview shows up early. Switch to interview mode: pause new learning for up to two weeks, do DSA mediums and a system-design drill tailored to that company, then resume.

Overload. Cut in this order: the optional Rice course, Grokking, extra CMU lectures, the second Linux course, depth in Phase 5. Never cut DSA, concurrency, system-design drills or capstone commits.

Prices and availability. Exam fees, Udemy library availability and student-pack offers change. Confirm before you commit money or time.

11. Course index and verification

Confirmed against the provider's page for this plan: Algorithms, Part I (Princeton, Coursera); Introduction to Git and GitHub (Google, Coursera); Introduction to Containers w/ Docker, Kubernetes & OpenShift (IBM, Coursera); Foundations of Linux and Linux Command Line (Packt, Coursera); AWS Cloud Technical Essentials (Amazon Web Services, Coursera); CMU 15-445/645 on YouTube.

Taken from the roadmaps you uploaded, which state they were checked on October 3, 2026: the Udemy, Pluralsight and DeepLearning.AI items, the Rice specialization, and the Total Seminars and Packt AWS specializations. Confirm each course's last-updated date and your library access before starting.

Free and interactive: Play with Docker, OverTheWire Bandit, Linux Journey, Learn Git Branching, GitHub Skills, Killercoda, Confluent Developer Kafka 101, Use The Index, Luke, HashiCorp Terraform tutorials, Hello Interview, ByteByteGo and Gaurav Sen on YouTube, NeetCode.